Analyzing the Role of AI-Assisted Programming in Modern Software Development

IT 3003 - Advanced Programming Techniques

Assignment 02 - AI-Assisted Programming

Submission Date: 14/07/2026

Submitted By: A. W. W. A. Nipun Lakshan

Registration Number: 2023s20371

Index Number: s17618



University of Colombo, Faculty of Science

Table of Contents

1. Introduction....4
2. What is AI-Assisted Programming?....4
3. Types of AI Coding Tools....5
01. Code Generation Tools....5
02. AI Debugging Assistants....5
03. Code Refactoring and Optimization Tools....6
04. AI Testing Assistants....6
4. Advantages of AI-Assisted Programming....7
01. Increased Development Productivity....7
02. Faster Problem Solving....7
03. Improved Learning Experience....7
04. Better Code Quality....7
05. Support for Rapid Prototyping....7
5. Risks and Limitations of AI-Assisted Programming....8
01. Hallucinated Code....8
02. Over Dependency on AI Tools....8
03. Security Risks....8
6. Problem Description....9
A. System Requirements....9
01. Vehicle Management Module....9
02. Vehicle Operations....10
03. Customer Management Module....10
04. Vehicle Rental Module....11
05. Vehicle Return Module....11
7.1 Version A: Traditional Programming Approach (Without AI Assistance)....12

References 12
7.2 Version B: AI-Assisted Programming Approach (Using AI Tools) 12
Prompt Message 01 13
Message 13
Prompt Message 02 13
Message 13
8. Comparison of Implementations 14
1. Development Time 14
2. Code Quality 14
3. Error Handling 15
4. Readability and Maintainability 15
9. UML Diagrams 16
1. Use Case Diagram 16
2. Class Diagram 17
3. Activity Diagram 17
10. Critical Analysis 18
1. Impact of AI on Coding Approach 18
2. Effect of AI on Understanding 18
3. Situations Where AI Should Not Be Used 18
4. Ethical Considerations in AI-Generated Code 19
11. Conclusion 19

1. Introduction

Artificial Intelligence (AI) has become an essential component of modern software engineering, influencing the way developers analyze problems, design solutions, write code, debug applications and improve software quality. With the advancement of AI - powered development tools such as code generation assistants, intelligent debugging systems, automated testing tools and programming support platforms, developers are now able to enhance productivity, reduce development time and explore new approaches to software development.

2. What is AI-Assisted Programming?

AI-Assisted Programming refers to the use of Artificial Intelligence technologies to support and enhance the software development process. These AI-powered tools help developers write, understand, improve and maintain software by providing intelligent suggestions, generating code snippets, identifying errors and automating repetitive programming tasks.

Unlike traditional programming, where developers manually write and debug every part of an application, AI-assisted programming allows developers to collaborate with intelligent systems that can analyze requirements, suggest possible solutions and accelerate development activities. However, AI tools are not replacements for software engineers. They act as supporting assistants that improve developer productivity while still requiring human supervision, decision-making and validation.

Modern AI-assisted programming tools are commonly based on large language models (LLMs) that have been trained on large amounts of programming-related data. These models can understand natural language instructions and generate programming solutions based on user requirements.

3. Types of AI Coding Tools

AI coding tools provide different types of assistance throughout the software development lifecycle.

01. Code Generation Tools

Code generation tools help developers create source code based on natural language descriptions or partial code inputs. Developers can describe the required functionality and the AI tool generates possible implementations.

Examples of usage:

1. Generating classes and methods
2. Creating database queries
3. Writing algorithms
4. Producing boilerplate code
5. Creating documentation comments

02. AI Debugging Assistants

AI debugging tools help developers identify and resolve programming errors. They can analyze error messages, examine code logic and suggest possible fixes.

Common uses include:

1. Explaining compiler errors
2. Finding logical mistakes
3. Suggesting solutions for runtime exceptions
4. Improving error-handling mechanisms

03. Code Refactoring and Optimization Tools

AI-powered refactoring tools help improve existing code without changing its functionality. They analyze code quality and suggest improvements.

Examples:

1. Reducing duplicated code
2. Improving variable and method names
3. Simplifying complex logic
4. Improving code readability
5. Suggesting better design patterns

04. AI Testing Assistants

AI testing tools support software testing activities by automatically generating test cases and identifying possible failure scenarios.

They can assist with:

1. Unit test generation
2. Test data creation
3. Identifying edge cases & Improving test coverage

4. Advantages of AI-Assisted Programming

01. Increased Development Productivity

AI tools can significantly reduce the time required for repetitive programming tasks. Developers can quickly generate basic code structures and focus more on solving complex business problems.

Example:

- Instead of manually creating getter and setter methods for multiple classes, developers can use AI tools to generate them instantly.

02. Faster Problem Solving

AI assistants can quickly analyze programming issues and provide possible solutions. This helps developers overcome technical challenges more efficiently.

03. Improved Learning Experience

For students and beginner programmers, AI tools can act as learning assistants by explaining concepts, providing examples and helping understand programming errors.

04. Better Code Quality

AI tools can identify potential improvements related to:

1. Code readability & Performance
2. Security practices
3. Programming standards

05. Support for Rapid Prototyping

Developers can quickly create prototypes and experiment with different approaches before implementing the final solution.

5. Risks and Limitations of AI-Assisted Programming

01. Hallucinated Code

AI tools may generate code that appears correct but contains errors, incorrect logic or outdated approaches. This happens because AI models generate responses based on learned patterns rather than true understanding of the problem.

Example:

- An AI tool may suggest a library function that does not exist or provide code that fails during execution.

02. Over Dependency on AI Tools

Excessive dependence on AI assistants can reduce developers' ability to independently analyze problems and design solutions.

Example:

- Students may focus on copying generated code instead of understanding programming concepts.

03. Security Risks

AI-generated code may introduce security vulnerabilities if it does not follow secure coding practices.

Possible issues include:

1. Weak authentication methods
2. Unsafe input handling
3. Exposure of sensitive information
4. Incorrect use of security libraries

Developers must review AI-generated code carefully before using it in real applications.

6. Problem Description

Premium Rental Services Management System

"**Premium Rental Services**" is a vehicle rental company that provides rental services for customers who need temporary transportation. The company owns different categories of vehicles such as cars, vans and motorcycles.

Currently, the company manages vehicle details, customer information and rental transactions manually. Due to the increasing number of customers and vehicles, the management wants to introduce a computerized system to efficiently handle daily rental operations.

Your task is to design and implement a prototype of a Vehicle Rental Management System for Premium Rental Services as a Java console - based application. The prototype should be developed using Core Java and Object - Oriented Programming (OOP) principles. All data should be stored temporarily in memory using Java Collections and database integration is not required. The system should demonstrate the core functionalities of vehicle rental management through a menu-driven console interface.

A. System Requirements

01. Vehicle Management Module

The system should maintain information about vehicles owned by the company. Premium Rental Services provides three types of vehicles:

1. Cars
2. Vans
3. Motorcycles

Each vehicle should contain common information:

1. Vehicle ID
2. Vehicle Name
3. Daily Rental Rate
4. Availability Status

The system should maintain additional details based on the vehicle type.

No	Car	Van	Motorcycle
01.	Number of Seats	Cargo Capacity	Engine Capacity
02.	Fuel Type	-	-

02. Vehicle Operations

The system should allow employees to perform the following operations:

1. Register a new vehicle
2. View all vehicles
3. Search vehicles by vehicle ID
4. Remove vehicles

03. Customer Management Module

Before renting a vehicle, customers must be registered in the system. The system should store:

1. Customer ID
2. Customer Name
3. Contact Number
4. Driving License Number

The system should provide the following operations:

1. Register customer
2. View all customers
3. Search customer by ID
4. Remove customer

04. Vehicle Rental Module

A registered customer can rent an available vehicle. The system should perform the following activities:

1. Verify whether the customer exists.
2. Verify vehicle availability.
3. Create a rental transaction.

Each rental transaction should contain:

1. Rental ID
2. Customer ID
3. Vehicle ID
4. Rental Start Date
5. Rental End Date
6. Number of Rental Days
7. Rental Estimated Cost [Daily Rate × Number of Rental Days]
8. Rental Discount
9. Rental Actual Cost
10. Rental Status

05. Vehicle Return Module

When a rented vehicle is returned. The system should:

1. Update rental status as completed.
2. Change vehicle availability status to available.
3. Store the completed rental record.

Premium Rental Services provides different discounts depending on the vehicle category.

No	Vehicle Type	Discount Percentage
01.	Car	5%
02.	Motorcycle	10%
03.	Van	8%

7.1 Version A: Traditional Programming Approach (Without AI Assistance)

Please refer to the first 17 commits starting from the commit stated as "**Initial Commit**" up to "**Revise README for clarity and project details**" to see the development stages in the GitHub link mentioned below for my traditional programming approach without AI assistance. In the folder called "**01. Manual Implementation (Version A)**" in the below GitHub repository link, you can see the whole working code developed through the traditional programming approach.

- GitHub Link: https://github.com/Nipun-Lakshan/car-rental-management-system.git

References

1. ITSC, IT 3003 *Advanced Programming Techniques: Lecture Notes*. University of Colombo, Faculty of Science, 2026.

2. GeeksforGeeks. (2026). *Java LocalDateTime Class*. https://www.geeksforgeeks.org/java/java-time-localdatetime-class-in-java/

3. W3Schools. (2026). *Java Date and Time*. https://www.w3schools.com/java/java_date.asp

7.2 Version B: AI-Assisted Programming Approach (Using AI Tools)

In this section, I implemented the entire project using different Large Language Models (LLMs) to evaluate whether they could generate the expected output. I have documented all the prompts used during the development process, along with the corresponding GitHub commits, to analyze the implementation workflow and assess the impact of AI-assisted coding on the development process. Please refer to the table below for a better understanding.

Prompt Message 01

- Tool Used: Claude AI
- Commit Number: e64801d
- Commit Message: Initial implementation using AI Prompt 01

Message

> Create a complete Java console-based application called "Premium Rental Services Management System" with seven classes following object-oriented principles. Start with an abstract Vehicle class containing private fields (vehicleId, vehicleName, dailyRentalRate, availabilityStatus) with constructor chaining where the final constructor initializes all fields using default values ("Unknown" for Strings, 0 for ints, false for booleans), plus public getters and setters. Then create three subclasses extending Vehicle: Car (with private int numberOfSeats, String fuelType, and static DISCOUNT_RATE=5), Motorcycle (with private int engineCapacity and static DISCOUNT_RATE=10), and Van (with private int cargoCapacity and static DISCOUNT_RATE=8), each implementing constructor chaining with super() and getters/setters for their additional fields. Additionally, create a Customer class with private fields (customerId, customerName, contactNo, licenseNumber) with constructor chaining and getters/setters, and a Rental class with private fields (rentalId, customerId, vehicleId, rentalStartDate, rentalEndDate, numberOfRentalDays, rentalEstimatedCost, discount, rentalActualCost, rentalStatus) also with constructor chaining and getters/setters. Finally, develop an Application class with a main method containing a cross-platform clear console function, a heading display showing "PREMIUM RENTAL SERVICES MANAGEMENT SYSTEM", and a main menu with options for vehicle management (register, view all, search by ID, remove), customer management (register, view all, search by ID, remove), and rental management (rent a vehicle, return a vehicle, view all rentals), plus an exit option, with all CRUD operations using ArrayLists or HashMaps for in-memory storage and a Scanner for user input within a do-while loop with switch-case navigation.

Prompt Message 02

- Tool Used: Claude AI
- Commit Number: b4cd0d8
- Commit Message: Implementation using AI Prompt 02

Message

> Write a more improve version of this code by not adding or removing existing things. Indees 7 files separately as above. [Attached ZIP file of my traditional programming files.]

8. Comparison of Implementations

The two implementations of the Car Rental Management System were evaluated based on development time, code quality error handling, readability and maintainability. The comparison highlights the differences between developing software with AI assistance and developing it using traditional programming approaches.

1. Development Time

The AI-assisted implementation required less development time compared to the traditional implementation. AI tools helped generate code structures, suggest solutions for programming problems and provide quick explanations for errors. This reduced the time required for writing repetitive code and debugging common issues.

In contrast, the traditional implementation required more time because all components including class structures, methods, validations and error-handling mechanisms had to be designed and implemented manually. However, the traditional approach provided more opportunities to understand the complete development process.

2. Code Quality

The AI-assisted implementation produced functional code quickly and provided suggestions based on common programming practices. It helped maintain consistency in coding style and reduced syntax-related errors. However, some generated code required manual review and modification to improve efficiency, remove unnecessary complexity and ensure it matched the project requirements.

The traditional implementation generally resulted in better understanding of the code structure because every component was designed by the developer.

3. Error Handling

In the AI-assisted implementation, error handling solutions were suggested quickly, especially for common programming issues such as input validation and exception handling. However, some cases required additional improvements because AI-generated solutions may not always consider all possible user inputs or project-specific scenarios.

The traditional implementation required more effort to identify and handle errors, but it allowed the developer to design error-handling mechanisms based on a deeper understanding of the system workflow. This resulted in more controlled and customized validation logic.

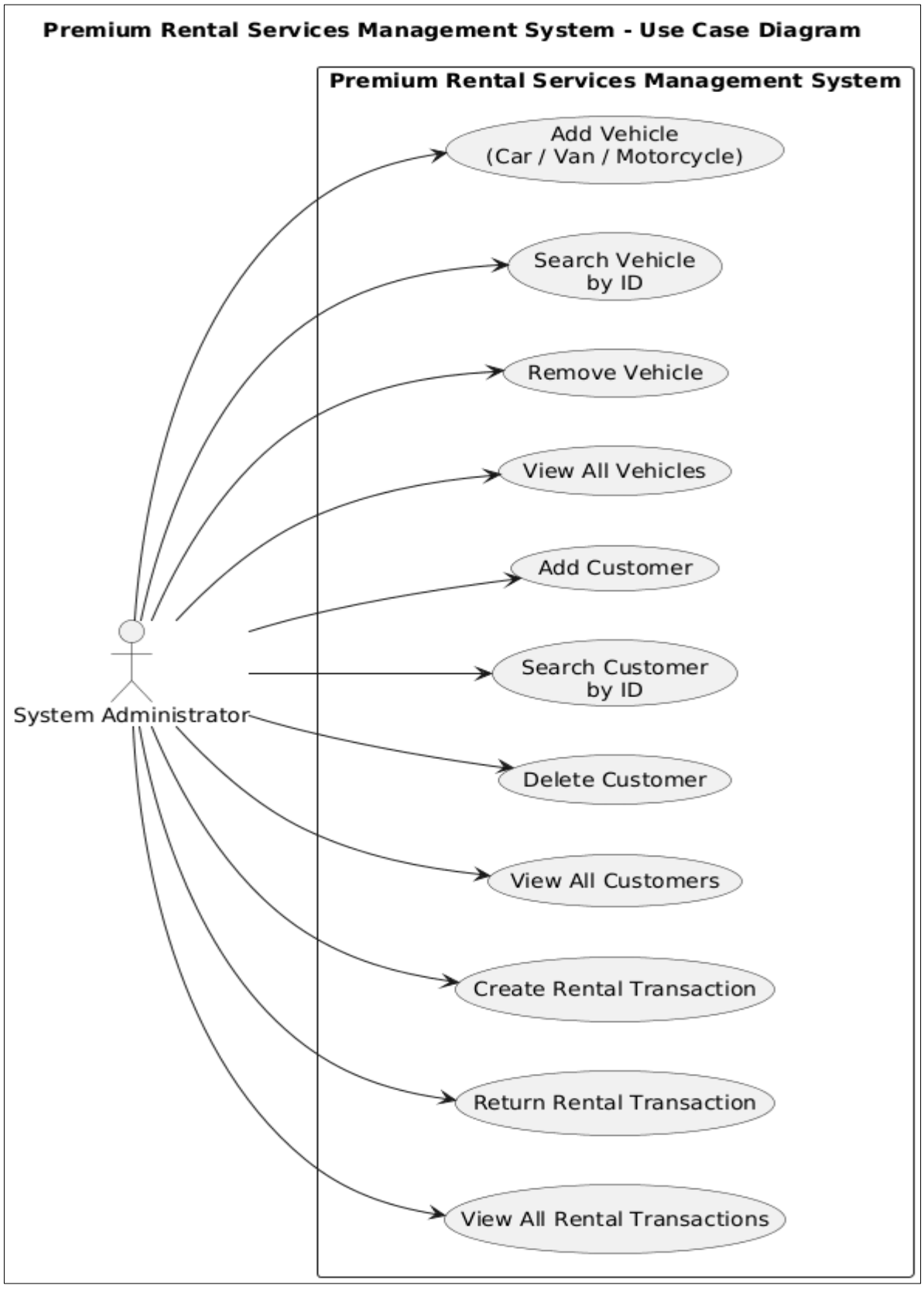
4. Readability and Maintainability

The AI-assisted implementation improved readability by suggesting structured code and meaningful naming conventions. However, maintaining the code required careful review because developers need to understand and verify AI-generated sections before making future modifications.

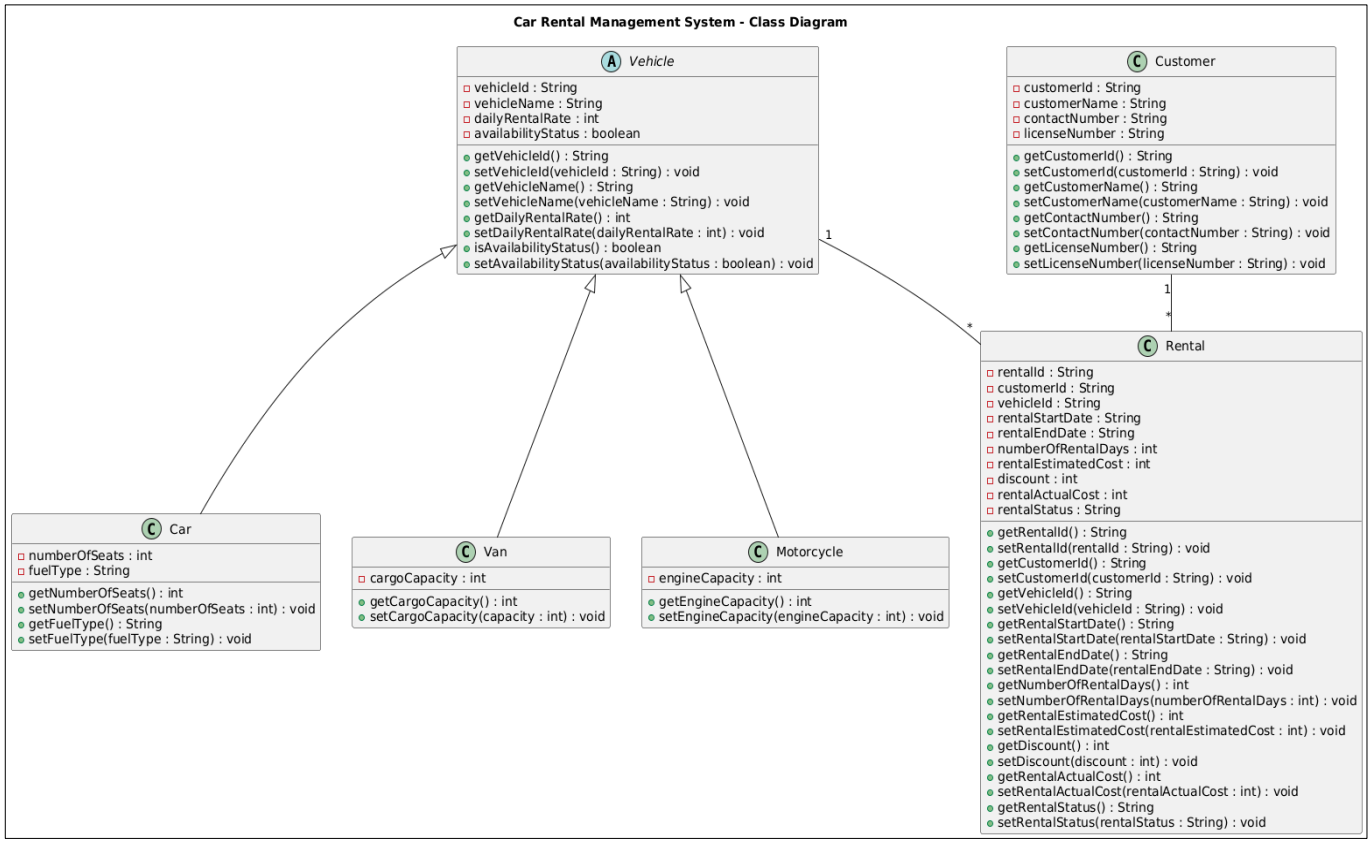
The traditional implementation provided better maintainability from the developer's perspective because the programmer had full knowledge of the design decisions and implementation logic. The code structure was created according to the developer's understanding of the problem, making future enhancements easier.

9. UML Diagrams

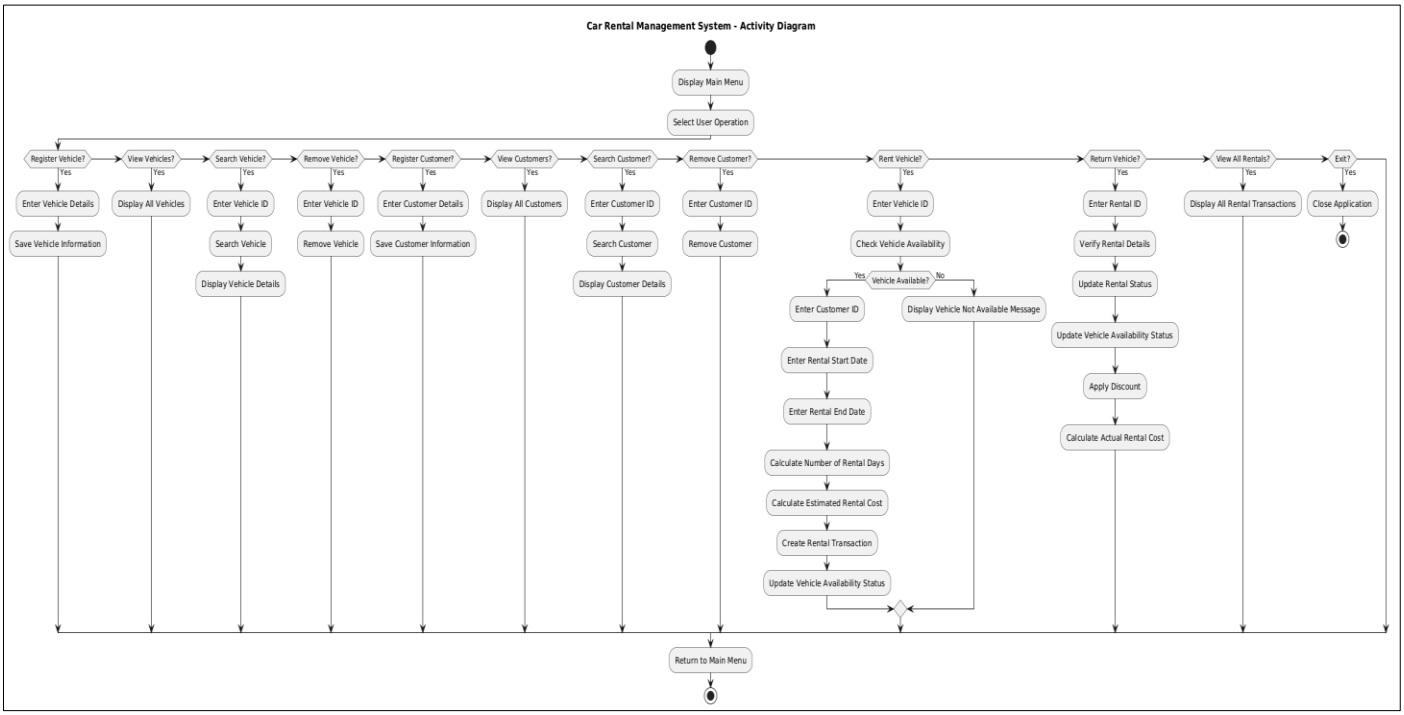
1. Use Case Diagram



2. Class Diagram



3. Activity Diagram



10. Critical Analysis

1. Impact of AI on Coding Approach

The use of AI tools changed coding approach by providing faster solutions, suggestions and alternative ways to implement features. Instead of writing every part of the program from scratch, AI helped generate code structures, identify errors and suggest improvements. This allowed more time to focus on understanding the system design, Object-Oriented Programming concepts and overall application logic.

However, AI-generated code still required manual checking and modification. Some generated solutions did not completely match the project requirements and additional adjustments were needed to improve functionality, readability and error handling.

2. Effect of AI on Understanding

AI had both positive and negative effects on understanding of programming. On the positive side, AI acted as a learning assistant by explaining programming concepts, providing examples and helping debug errors. It was especially useful for understanding unfamiliar concepts and improving code quality.

On the other hand, depending too much on AI-generated solutions could reduce the opportunity to practice problem-solving skills. If code is copied without understanding the logic behind it, the learning process can be negatively affected. Therefore, AI should be used as a supporting tool rather than a replacement for learning programming fundamentals.

3. Situations Where AI Should Not Be Used

AI should not be used as a complete replacement for a programmer's own thinking and decision-making. It should be avoided in situations where;

1. The generated code is used without understanding its functionality.
2. Sensitive information or confidential project details are shared with AI tools.

4. Ethical Considerations in AI-Generated Code

When using AI for programming, several ethical issues should be considered;

01. Code Ownership: Developers should understand licensing and ownership concerns related to AI-generated code.

02. Transparency: Users should disclose when AI tools are used in software development when required.

03. Security Risks: AI-generated code may contain vulnerabilities, so security reviews are necessary.

04. Academic Integrity: Students should avoid submitting AI-generated work as their own without understanding or acknowledgment.

05. Bias and Accuracy: AI suggestions may contain errors or outdated practices, so human judgement is required.

11. Conclusion

AI has become a valuable tool in programming education and the software industry by improving productivity, reducing development time and assisting developers in solving technical problems. In programming education, AI can support students by providing explanations, examples and debugging assistance. However, it should be used responsibly because excessive dependency on AI may reduce independent problem-solving skills.

In the software industry, AI can improve development efficiency by assisting with code generation, testing, documentation and debugging. However, human developers are still essential for system design, decision-making, security evaluation and maintaining software quality.

Overall, AI should be considered a supportive technology rather than a replacement for programmers. The most effective approach is combining AI assistance with human creativity, critical thinking and programming knowledge to produce reliable and high-quality software solutions.